UNIX 系統程式設計期末專題提案書

BusyBox 系統診斷工具集：輕量化 Linux 系統監控與健康檢測工具

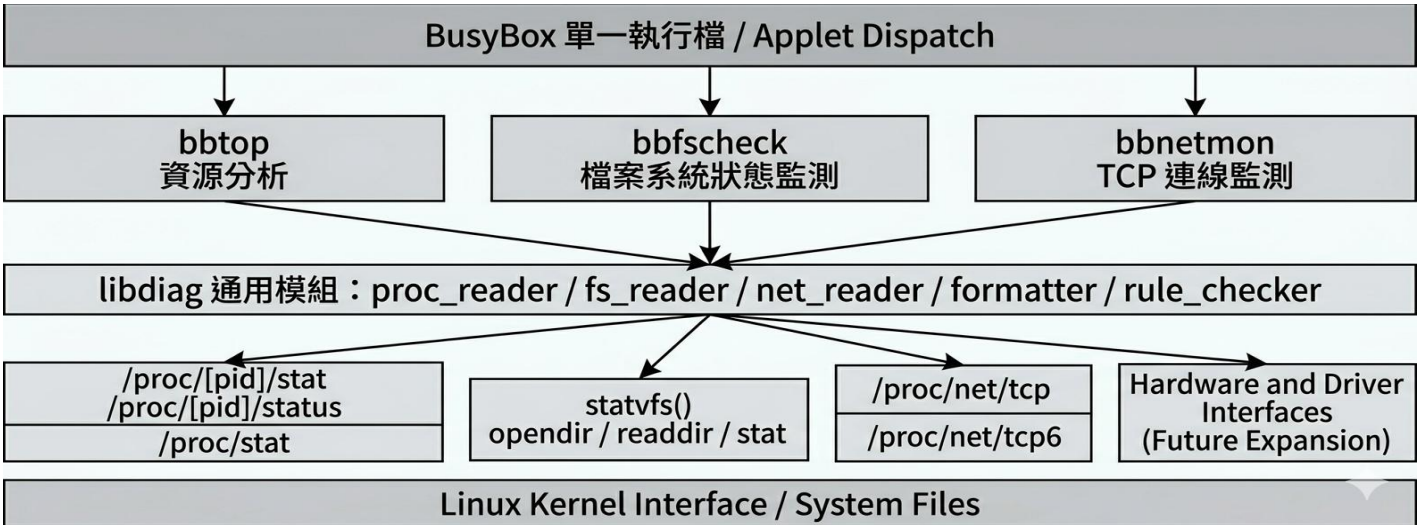
選定方向：選項 B－BusyBox 工具擴充／方向一：系統診斷工具集

一、動機與背景

在 Linux 系統管理、嵌入式設備維護與伺服器除錯中，開發者常需要快速掌握行程、檔案系統與網路連線狀態。雖然一般環境有 top、df、du、ss 等工具，但在嵌入式或最小化系統中，套件可能不完整、執行檔體積較大或功能分散。本專題希望基於 BusyBox applet 架構，實作一組輕量化系統診斷工具，並透過共用資料收集模組降低重複程式碼，實際練習 UNIX/Linux 系統程式設計中的 /proc 解析、檔案系統查詢、TCP 狀態分析與效能評估。

二、系統架構與模組劃分

三個工具將整合為 BusyBox 單一執行檔中的 applet，並抽出 libdiag 作為共用內部函式庫，負責資料讀取、共用資料結構、錯誤處理、表格輸出與健康規則判斷。整體資料流由 BusyBox applet 接收命令列選項，再呼叫 libdiag 讀取 Linux kernel 暴露的 /proc、statvfs() 與 TCP 狀態資料，最後輸出診斷結果與 Benchmark 報告所需數據。



BusyBox applet	主要功能	資料來源／系統呼叫	共用模組
bbtop	行程資源分析、CPU/Memory 統計、PID tree 顯示	/proc/[pid]/stat、/proc/[pid]/status、/proc/stat	proc_reader、formatter
bbfscheck	磁碟容量、inode 使用率、目錄小檔案分布與健康狀態	statvfs()、opendir()、readdir()、stat()	fs_reader、rule_checker
bbnetmon	TCP 連線列表、state 統計、異常連線偵測	/proc/net/tcp、/proc/net/tcp6	net_reader、formatter

三、預期功能與技術方案

工具	預期功能	技術方案與最佳化方向
bbtop	提供 snapshot 與 interval 模式，顯示 PID、PPID、指令名稱、CPU%、Memory，支援依 CPU/Memory 排序與 PID tree。	提供 snapshot 與 interval 模式，顯示 PID、PPID、指令名稱、CPU%、Memory，支援依 CPU/Memory 排序與 PID tree。讀取 /proc/[pid]/stat、/proc/[pid]/status 與 /proc/stat；以兩次 sampling 計算 process/system time delta；建立 PID→PPID 對應產生樹狀輸出；benchmark 模式避免互動式 UI 與不必要 name lookup。
bbfscheck	提供 summary、inode 與 scan 模式，顯示掛載點容量、inode 使用率，並掃描指定目錄找出大量小檔案或 inode 壓力。	使用 statvfs() 取得 block/inode 資訊；以 opendir/readdir/lstat 逐項統計檔案數、總大小與平均大小；提供 --max-depth 與 skip /proc、/sys、/dev，避免掃描虛擬檔案系統造成誤判。
bbnetmon	列出 TCP local/remote address、port、state，統計 ESTABLISHED、LISTEN、TIME_WAIT、	解析 /proc/net/tcp 與 /proc/net/tcp6 的十六進位欄位；以 table mapping 轉換 TCP state；summary 模式採單次掃描直接統計，不做 DNS/service name 反查，以降低輸出與解析成本。

	CLOSE_WAIT 等狀態，並支援 state 篩選與異常提示。	

四、分工說明

組員	學號	主要負責項目	交付成果
組員 A	S1063042	bbtop、/proc 行程資料解析、CPU/Memory 計算	行程監控 applet、PID tree demo
組員 B	S1154005	bbfscheck、statvfs 與目錄掃描	檔案系統健康檢測與測試資料
組員 C	S1154009	bbnetmon、TCP state 解析與異常規則	網路監控 applet、連線狀態 demo
組員 D	S1154035	BusyBox 整合、libdiag、測試、Benchmark、文件	可編譯版本、README、man page、報告簡報

五、預計時程

時間	工作內容
4/30 前	完成提案、確認工具名稱、CLI 設計、模組劃分與分工。
5/1 - 5/5	研究 BusyBox applet 與 build system，完成 hello applet 測試。
5/6 - 5/10	完成 libdiag 初版與三個工具的資料讀取 prototype。
5/11 - 5/16	完成 bbtop、bbfscheck、bbnetmon 核心功能。
5/17 - 5/20	整合 BusyBox、修正 CLI、錯誤處理與輸出格式。
5/21 - 5/23	進行功能測試與 Benchmark，與 top/df/du/ss 等工具比較。
5/24 - 5/26	完成 README、CONTRIBUTING、man page、書面報告、簡報與 Demo。

六、預期困難與解決方案

可能困難	解決方案
BusyBox applet 與 build system 不熟悉	先建立最小 hello applet 驗證編譯流程，再逐步加入正式工具，降低整合風險。
/proc 格式複雜且欄位易解析錯誤	將 parser 集中於 libdiag，使用實際系統輸出與 ps、ss 等工具交叉比對。
CPU 使用率需透過 sampling 計算	設計 interval 機制，比較前後兩次 process/system time delta。
大量目錄掃描可能影響效能	提供指定目錄與深度限制，避免預設掃描整個系統；以 Benchmark 分析瓶頸。
與 GNU/util-linux 工具功能不完全相同	選擇功能相近情境進行比較，報告中清楚說明測試條件與功能範圍。
之前沒設計過 Benchmark 測試，可能產生不公平比較	將每個工具拆成可比較模式：bbtop --snapshot 對 ps/top batch、bbfscheck --summary 對 df/df -i、scan 對 du/find、bbnetmon --summary/list 對 ss/netstat；統一測試環境、多次執行取中位數，並記錄時間、記憶體與輸出量。

七、預期成果

本專題預期完成一個可正常編譯與執行的 BusyBox 擴充版本，包含 bbtop、bbfscheck、bbnetmon 三個互相整合的系統診斷 applet，以及 libdiag 共用模組。最終將提供 GitHub 原始碼、README、CONTRIBUTING、man pages、功能測試、Benchmark 報告與 Demo 展示，展示在實際 Linux 環境中進行行程、檔案系統與網路狀態診斷的完整流程。Benchmark 報告將列出各工具的測試模式、對照工具、原版時間、本工具時間、記憶體使用量、效能差距與最佳化說明，目標是在相同功能範圍下將效能差距控制於原版工具 50% 以內